Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

List and data classes will definitely be useful in this project. Storing information, that being recipes, is the whole point of the project, with the recipes having different information I could use dataclasses to represent them, and lists to represent the different recipes. Improving on loops and conditionals is a goal of mine, as I didn't do the best on the exams which covered those two topics.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

Lists and dataclasses, I did somewhat well on the exam which covered those topics and I enjoy the information aspect of it. On a more technical side, recipes fit into a dataclass, with all the recipes having a field and information in that field.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Actually implementing the drafter features will take the most time and effort. I want to implement the “Recipe” dataclass and the “State” dataclass first. (all recipes need information to be stored in their dataclass, while the state dataclass ‘remembers’ it)

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

No distractions. I think I'm all set for this project and pretty excited to see what I’m going to make.

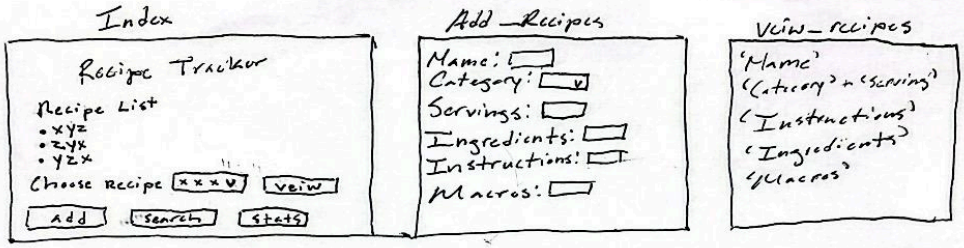
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

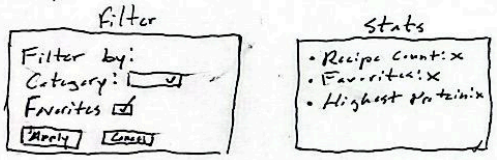
We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

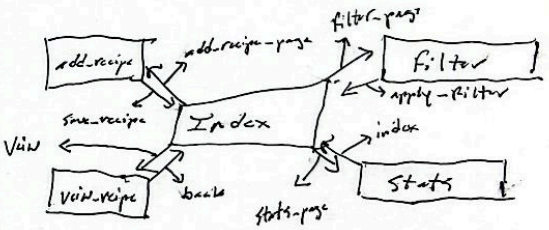


Simple Mock Up



State Dataclass	Recipe Dataclass
Recipes: list[Recipe] Message: str Selected: int Category_filter: str Favs_only: bool	name: str category: str servings: int ingredients: str instructions: str cost: float Macros: int Fav: bool

*Each box is a page, I really only wrote them to explain to myself the diagram on the bottom right but it's been referenced since I began on coding *



*I'm sorry it looks like this, for arrows off arrow tip just to show their point better *

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**My app will allow users to add recipes and related information into a program, with the option to sort them based off category or favorites.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	**State Dataclass** recipes: list[Recipe] → Stores all saved recipes, allowing for them to be added to or deleted message: str → Displays feedback or error message to user selected: int → Stores the index of the currently selected recipe in 'recipes' category_filter: str → Stores the selected category filter favorites_only: bool → Determines whether the category filter displays only favorited recipes
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	**Recipe Dataclass** ******All information about what the individual recipe will have, ex: their name****** Name:str category: str servings: int ingredients: str instructions: str cost:float **macros: list[float]**

	favorite: bool
Routes (minimum 7) The names of your route functions	***All on bottom right of drawing*** index add_recipe save_recipe view_recipe recipe_page toggle_favorties delete_recipe filter_page apply_filter stats_page
Pages (minimum 5) A brief description of the pages in your site	index → Home page, displays the recipes and actions to go to the other pages add_recipe → Allows users to enter recipes and their information view_recipes → Displays the information for the selected recipe, ex: name, ingredient filter → Filter recipes by a category or favorites stats → Displays recipe count, favorites, highest protein, *not really 100% what I want to do here?*
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	index → SelectBox: Allows user to select a recipe to view add_recipe → TextBox(s): Allows users to enter name, severings, and macros add_recipe → TextArea(s): Allows user to enter ingredients and instructions add_recipe → SelectBox: Allows user to select a category filter → SelectBox: Allows users to filter by a category filter → CheckBox: Allows user to filter by favorites
If Statements (minimum 3)	**If a recipe name is blank, return an error**

Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	**message. This will prevent any empty recipes** **If a recipe category is equal to the category of the filter, display that recipe. This will be the logic of showing the recipes which share the category of the filter** **If a number in the macros section is below 0, return an error message. This will prevent illogical input.**
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	A loop that checks over each recipe's category against the current filters. In other words, the loop is checking each recipe to see if it matches the filter, being the logic for the filtering of the program.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Anything which is in bold was put down with the help of the chat below

I'm creating a SIMPLE app which allows users to store recipes and filter them for later use. How could I encorperate If Statements into my code? Heres a rough draft of what I have so far:

Requirement	Implementation
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**My app will allow users to add recipes and related information into a program, with the option to sort them based off category or favorites.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	**State Dataclass** recipes: list[Recipe] → Stores all saved recipes, allowing for them to be added to or deleted message: str → Displays feedback or error message to user selected: int → Stores the index of the currently selected recipe in ‘recipes’ category_filter: str → Stores the selected category filter favorites_only: bool → Determines whether the category filter displays only favorited recipes

Second Dataclass	**Recipe Dataclass**
Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	******All information about what the individual recipe will have, ex: their name****** Name:str category: str servings: int ingredients: str instructions: str cost:float macros: list[float] favorite: bool
Routes (minimum 7) The names of your route functions	***All on bottom right of drawing*** index add_recipe save_recipe view_recipe recipe_page toggle_favorties delete_recipe filter_page apply_filter stats_page
Pages (minimum 5) A brief description of the pages in your site	index → Home page, displays the recipes and actions to go to the other pages add_recipe → Allows users to enter recipes and their information view_recipes → Displays the information for the selected recipe, ex: name, ingredient filter → Filter recipes by a category or favorites

	stats → Displays recipe count, favorites, highest protein, *not really 100% what I want to do here?*
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see .	index → SelectBox: Allows user to select a recipe to view add_recipe → TextBox(s): Allows users to enter name, severings, and macros add_recipe → TextArea(s): Allows user to enter ingredients and instructions add_recipe → SelectBox: Allows user to select a category filter → SelectBox: Allows users to filter by a category filter → CheckBox: Allows user to filter by favorites

Your app is actually a great opportunity to incorporate `if` statements because recipes naturally have conditions that determine what the program should display or do.

Since you're keeping the app simple, I wouldn't add unnecessary features just to use conditionals. Instead, incorporate them into the functionality you already have.

Here are **6 ways you could use `if` statements** in your app.

1. Adding a recipe (input validation)

When a user submits a recipe, check whether they've entered the required information.

None

```
if recipe_name == "":    message = "Please enter a recipe name."else:    # Save the recipe recipes.append(new_recipe)    message = "Recipe successfully added!"
```

Purpose: Prevents users from saving recipes without a name.

You could also check whether servings are greater than zero.

```
None
if servings <= 0:    message = "Servings must be greater than zero."
```

2. Filtering by category

When a user selects a category, display only recipes belonging to that category.

```
None
if recipe.category == category_filter:
filtered_recipes.append(recipe)
```

Purpose: Allows users to narrow down their recipes based on categories like desserts, breakfast, dinner, etc.

You could also account for an "All" option:

```
None
if category_filter == "All":    filtered_recipes = recipes
```

3. Filtering favorites

You already have a `favorites_only: bool` attribute. This is probably one of the best places to incorporate conditionals.

```
None
if favorites_only == True:    if recipe.favorite == True:
filtered_recipes.append(recipe)else:
filtered_recipes.append(recipe)
```

Purpose: If the checkbox is selected, only favorited recipes appear. Otherwise, all recipes appear.

You can simplify this further using Boolean logic, but the nested conditionals make it easy to understand.

4. Deleting recipes

Before deleting a recipe, check whether the selected recipe actually exists.

```
None
if 0 <= selected < len(recipes):    recipes.pop(selected)
message = "Recipe deleted!"else:    message = "Please select
a valid recipe."
```

Purpose: Prevents errors when users attempt to delete a recipe that doesn't exist.

5. Displaying recipe information

You could use conditionals to display certain information depending on the recipe.

For example, displaying whether a recipe is a favorite:

```
None
if recipe.favorite:    message = "★ Favorite Recipe"else:
message = "☆ Not a Favorite"
```

You could also use this for nutritional information:

```
None
if recipe.macros[0] >= 30:    message = "High Protein
Recipe"else:    message = "Regular Protein Recipe"
```

(Assuming `macros[0]` represents protein.)

6. Stats page

Your stats page could actually be pretty useful without becoming overly complicated.

Some possible statistics:

- Total number of recipes.
- Number of favorited recipes.
- Number of recipes in each category.
- Highest-protein recipe.

You could use conditionals to identify the highest-protein recipe:

None

```
if recipe.macros[0] > highest_protein:    highest_protein = recipe.macros[0]    highest_protein_recipe = recipe.name
```

My recommendation: Where to incorporate conditionals

For your project, I'd prioritize these four:

Feature	Conditional	Purpose
Add Recipe	`if recipe_name == ""`	Prevents empty recipes
Filter Recipes	`if recipe.category == category_filter`	Filters by category
Favorites	`if recipe.favorite == True`	Displays favorited recipes
Delete Recipe	`if 0 <= selected < len(recipes)`	Prevents invalid deletions

These are all naturally integrated into your existing design.

One suggestion for your dataclasses

I actually think your current attributes are pretty good. I'd keep them largely as they are:

State Dataclass

- `recipes: list[Recipe]`
- `message: str`
- `selected: int`
- `category_filter: str`
- `favorites_only: bool`

Recipe Dataclass

- `name: str`
- `category: str`
- `servings: int`
- `ingredients: str`
- `instructions: str`
- `cost: float`
- `macros: list[float]`
- `favorite: bool`

One small consideration: I'd make `macros` a list of floats representing protein, carbohydrates, and fat, in that order. That gives you a consistent way to access nutritional information.

Overall: You don't need to create additional pages or attributes to demonstrate `if` statements. Your existing add, delete, filter, and favorites functionality already provides plenty of opportunities to use them.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.